Lista 2
Algoritmos em grafos
Prof. J.D.

1. Prove que $\sum_{i=0}^{d} a_i x^i = O(n^d)$ onde $a_i \in \mathbb{R}$ para todo i e $a_d > 0$.

2. Prove que $n\log(n) + O(n) = O(n\log(n))$.

3. Sejam G e H grafos isomorfos e $f\colon G \to H$ um isomorfismo. É verdade que $G[U]$ é isomorfo a $H[f(U)]$ para todo $U \subseteq V(G)$? Justifique.

4. Seja $G = (V, E)$ um grafo com $|V| = n$ e $|E| = m$.

   (a) Mostre que uma representação de G por matriz de adjacência utiliza $\Theta(n^2)$ posições, enquanto uma representação por listas de adjacência utiliza $\Theta(n + m)$ posições.

   (b) Para cada uma das situações abaixo, indique qual representação é mais adequada, matriz ou listas de adjacência, e justifique sua escolha comparando o custo das operações relevantes.

   - $m = \Theta(n)$ e o algoritmo precisa percorrer todos os vizinhos de cada vértice.
   - $m = \Theta(n^2)$ e o algoritmo realiza muitas consultas do tipo "$i, j \in E$?".
   - $m = \Theta(n)$ e o algoritmo precisa apenas escrever todas as arestas.
   - $m = \Theta(n^2)$ e o algoritmo precisa escrever todas as arestas.

   (c) Explique por que, apesar das diferenças de eficiência, as duas representações não alteram a classificação de um algoritmo como sendo de tempo polinomial.

5. Considere uma execução da busca rotulada $\mathrm{BP}(G, w)$ (veja notas de aula) e defina, para todo $t \in \mathbb{N}$, a t-ésima iteração do vetor p, denotada por $p^{(t)}$, recursivamente, da seguinte maneira

$$p^{(t)}[v] = \begin{cases} v & \text{se } t = 0 \\ p^{(t-1)}\big[p[v]\big] & \text{se } t > 0. \end{cases}$$

   Prove as seguintes equivalências: para quaisquer $u, v \in V(G)$

   (a) $\big[\mathrm{chega}(v), \mathrm{sai}(v)\big] \cap \big[\mathrm{chega}(u), \mathrm{sai}(u)\big] = \varnothing$ se e somente se não existe $t \in \mathbb{N}$ tal que $p^{(t)}[u] = v$ e não existe $t' \in \mathbb{N}$ tal que $p^{(t')}[v] = u$;

   (b) $\big[\mathrm{chega}(v), \mathrm{sai}(v)\big] \subset \big[\mathrm{chega}(u), \mathrm{sai}(u)\big]$ se e somente se existe $t \in \mathbb{N}$ tal que $p^{(t)}[v] = u$;

   (c) $\big[\mathrm{chega}(v), \mathrm{sai}(v)\big] \supset \big[\mathrm{chega}(u), \mathrm{sai}(u)\big]$ se e somente se existe $t \in \mathbb{N}$ tal que $p^{(t)}[u] = v$.

6. Sejam $G = (V, E)$ um grafo e $L \subseteq V$ um conjunto de vértices. O conjunto L define um ***corte de arestas*** em G, denotado $\nabla_G(L)$ e dado por

$$\nabla_G(L) = \big\{\{u, v\} \in E\colon u \in L,\ v \in \overline{L}\big\}$$

   onde $\overline{L} = V \setminus L$.

   Considere o seguinte algoritmo sobre um grafo conexo G

Dado : um grafo conexo G.
Devolve : visita aos vértices de G.

1 marque todos os elementos de $V(G)$ *não-visitado*;
2 $M = \varnothing$;
3 escolha um vértice v *não-visitado*;
4 insira v em L e marque v *visitado*;
5 **enquanto** $\nabla_G(L) \neq \varnothing$ **faça**
6 | escolha uma aresta $\{u, w\} \in \nabla(L)$;
7 | insira $\{u, w\}$ em M;
8 | $v \leftarrow \{u, w\} \cap \overline{L}$// v `é o extremo em` $\overline{L}$
9 | insira v em L;
10 | marque v *visitado*.

Algoritmo 1: Visite_vértices (G).

(a) O algoritmo visita todos os vértices de G? (Pode-se assumir que $\nabla(L) = \varnothing$ se, e só se, $L = V(G)$ – dado que $L = \varnothing$ ao longo de toda a execução – e esse fato será provado em um capítulo adiante).

(b) Prove que, no final de execução do algoritmo, M contém $|V(G)| - 1$ arestas.

(c) Determine a complexidade do algoritmo, propondo uma estrutura de dados que permita testar $\nabla(L) \neq \varnothing$ e escolher uma aresta de $\nabla_G(L)$ eficientemente. Pode-se assumir que as demais operações em L e em M são feitas em tempo constante.

(d) Prove que $G[M]$ não contém circuito (dica: indução no número de iterações: cada aresta inserida em M conecta um vértice já em L a um vértice novo (que nunca tocou M antes, pois estava em $\overline{L}$)).